

Module 0 — Computing, Data, and Reproducibility: Storage-Aware Batch Processing

Curriculum module: [Module 0 — Computing, Data, Statistics, and Reproducibility](#). Guiding question: *how do I set up a computing environment and habits that make my analysis trustworthy and repeatable before I even touch biological data?* This module's scripts live in [scripts/](#); the lecture deck is in [lecture/](#). All commands below assume you `cd` into `scripts/` first (or prefix each filename with `scripts/`).

This module is a five-step progression designed to be run **in order**. Each script adds exactly one new idea on top of the previous one, so the concepts build gradually before students move on to the larger capstone pipelines (AMR, RNA-seq, SNP calling, microbiome, etc.) elsewhere in this project.

A sixth script and a companion helper extend the module with the same idea applied to a second HPC resource — memory — once the five-script storage progression is complete. See **Bridge Extension: Predicting Memory Requirements** below.

Scripts 7–10 close the rest of this module's curriculum scope — environment specification, Git-based version control, file-format validation, and an integrated capstone artifact — using the same one-new-idea-at-a-time approach. See **Extension: Environments, Git, File Formats, and the Integrated Capstone** further down.

#	Script	New idea introduced
1	<code>1_The_Hardcoded_Script_SingleRun.sh</code>	Run one sample, start to finish, with nothing hidden
2	<code>2_Automated_Script_-_Processes_ALLRuns.sh</code>	Loop over a list of samples — full automation, no safety net
3	<code>3_Automated_Script_Hardcoded_20GB_Limit.sh</code>	Add a storage guard, but hardcode the limit deep in the script
4	<code>4._Automated_Script_User-DefinedMaximumSpace.sh</code>	Move the limit to a clearly labeled settings block anyone can find
5	<code>5_Automated_Script_System-BoundDynamicLimit.sh</code>	Stop guessing your own usage — ask the filesystem directly how much space is actually left
6 (ext.)	<code>6_Predicting_Memory_Requirements.sh + suggest_mem_from_history.sh</code>	Apply the same "measure, don't guess" habit to memory: log real usage, then predict the next run's — mem from history
7 (ext.)	<code>7_Environment_Reproducibility.sh</code>	Pin the exact software toolchain in a portable, version-locked <code>environment.yml</code> — not just an HPC module <code>load</code>

#	Script	New idea introduced
8 (ext.)	8_Git_Reproducibility_Workflow.sh	Pin the exact code version with a Git commit hash, embeddable in any run log
9 (ext.)	9_File_Format_Validation.sh	Validate that FASTA/FASTQ/SAM/BAM/BED/GFF/GTF/VCF files actually have the structure they claim to, before and after trusting them
10 (ext.)	10_Integrated_Foundations_Capstone.sh	Combine every habit above into one run that produces this module's required portfolio artifact, including a reproducibility reflection

Corrections applied to every script (2–5)

These four fixes were needed in every automated (loop-based) script you uploaded, so they are applied consistently across all four rather than just once:

- 1. Missing `srr_list.txt` check.** The original scripts assumed the list file exists and has content. If it didn't, `cat srr_list.txt` failed silently and the loop ran zero times — producing a confusing *empty* MultiQC report with no explanation. Every script now checks the file exists and is non-empty before doing anything else, and exits with a clear error message if not.
- 2. Fragile for `SRR` in `$(cat srr_list.txt)` loop.** This pattern word-splits the entire file, which silently breaks on a trailing blank line or on Windows-style line endings (`\r\n`) — a very common issue if the list was ever edited in Excel or Notepad. Every script now uses a `while IFS= read -r SRR` loop, which reads exactly one line at a time and strips a stray `\r` if present. This is the standard, safe way to loop over a file in bash.
- 3. No check that the download actually worked.** None of the original scripts verified that `fasterq-dump` produced real output before handing it to `fastqc/fastp`. A failed download (bad accession, dropped connection, expired SRA mirror) would cascade into confusing tool errors. Every script now checks that both paired FASTQ files exist and are non-empty; on failure it logs the accession to `failed_downloads.txt` and moves on to the next sample, instead of stopping the whole batch.
- 4. Cluttered output folder.** Every script previously wrote each sample's `fastp` HTML report directly into the shared `module1/working` directory. Once you're processing dozens of samples, that folder becomes unreadable. Reports now go into their own `fastp_reports/` subfolder.

Script-specific corrections

- **Script 1 (single run):** added a fail-fast check after the download — if the one sample fails, the script stops immediately with a clear message instead of continuing to run QC on nonexistent files.
- **Script 3 (hardcoded 20GB limit):** added a numeric sanity check on the `du`-derived `$CURRENT_GB` value before using it in the `-ge` comparison, so an unexpected `du` output format produces a warning instead of a hard script crash (integer expression expected).
- **Script 4 (user-defined limit):** same numeric sanity check as script 3, plus a factual correction to the comment about overshoot risk. The original note said the folder might exceed the limit by only "1-2GB" — that's only true for small runs. The real worst case is bounded by your *single largest run* in the list (which can be 5-10GB+ for typical paired-end data), so the corrected note tells students to leave headroom accordingly rather than assuming a fixed small overshoot.
- **Script 5 (system-bound dynamic limit):** same numeric sanity check on `$AVAILABLE_GB`, plus a new nuance note explaining that `df` reports space free on the *entire shared filesystem*, not the student's personal quota — so on quota-enforced systems, a `df`-based check (script 5) and a quota-based check (scripts 3/4) protect against two different limits, and robust production pipelines generally need both, not either/or.

What was intentionally left unchanged

- The Slurm `#SBATCH` resource requests (`--time`, `--mem`, `--cpus-per-task`) were left as originally set — they already scale sensibly across the series (2h for one sample → 10h for a full batch → 24h once the script can run until the disk is nearly full).
- The core tool commands (`fasterq-dump`, `fastqc`, `fastp`, `multiqc`) and their flags are unchanged — the corrections are about control flow and safety, not the underlying bioinformatics.
- Script 3's decision to hardcode `LIMIT_GB=20` in the middle of the file (rather than in a labeled settings block, like script 4 does) was kept as-is on purpose — that contrast between scripts 3 and 4 is itself the lesson about writing reusable code, and is explained in the header comment of script 3.

Building your own `srr_list.txt`

Scripts 2–6 expect a file named `srr_list.txt` in the same directory, containing one SRA run accession per line, with no header row and no blank lines, for example:

```
SRR11092056
SRR11092057
```

Bridge Extension: Predicting Memory Requirements

Scripts 1–5 are entirely about one HPC resource: **storage**. Slurm enforces a second resource just as strictly — **memory** — and a job that exceeds its requested `--mem` is killed immediately (State: `OUT_OF_MEMORY`), no matter how much wall-clock time was left. The same "measure, don't guess" instinct from script 5 applies here, just aimed at a different number.

There are three ways to predict memory, from lightest to most rigorous:

1. **Tool-specific formulas (before running anything).** Some tools document a direct relationship between input size and RAM — e.g. STAR needs roughly 10× the reference genome size in bytes for its index (~30GB for the human genome). Good for a starting `--mem` guess with no history at all.
2. **Historical accounting (what script 6 does).** Every completed Slurm job has real memory-usage data sitting in the accounting database. `seff $SLURM_JOBID` and `sacct -j <jobid> -o reqmem,maxrss,avrss` surface it. Script 6 automatically appends its own `seff` summary to `mem_usage_log.txt` as its last step, turning every run into a data point for sizing the next one.
3. **Automated prediction from that history** (`suggest_mem_from_history.sh`). Once a few runs are logged, this companion helper queries `sacct` for every completed job with a given name, takes the highest `MaxRSS` observed, adds a safety buffer (default 25%), and prints a recommended `--mem` value for your next submission — run it on the login node (**not** with `sbatch`) between batches:

```
bash scripts/suggest_mem_from_history.sh qc_mem_predict 25
```

Nuances specific to this extension

- **seff's sampling interval.** Usage is sampled periodically (roughly every 5 minutes by default), so it can under-report peak memory for very short jobs or brief spikes between samples.
- **MaxRSS is per task/node, not automatically summed.** For multi-task or multi-node jobs, you need to account for every task, not just the busiest one, when translating `MaxRSS` into a `--mem` request.
- **The bootstrap problem.** The very first run of script 6 has no history to learn from — that's why its own `--mem=12G` came from a tool-specific/file-size heuristic (method 1), not from `suggest_mem_from_history.sh`. Only switch to the historical helper once a few runs have completed.

- **A prediction, not a guarantee.** `suggest_mem_from_history.sh` extrapolates only from the accessions and read depths already processed. A batch with noticeably larger or smaller samples should treat the suggestion as a starting point — exactly the same caveat that applied to script 4's user-defined storage limit.

Extension: Environments, Git, File Formats, and the Integrated Capstone

Scripts 1–6 cover two of this module's curriculum facets — **compute environments** (storage, memory) and **responsible practice** (validate before trusting). The curriculum's full Module 0 scope (see [docs/curriculum/modular_bioinformatics_curriculum.md](#)) also calls for environment specification (Conda/Mamba), Git-based version control, and file-format literacy — culminating in a single portfolio artifact: *"a repository containing a small pipeline, environment specification, README, input checks, log, outputs, and a reproducibility reflection."* Scripts 7–10 close that gap, continuing to use the **PRJNA1518998** mouse retina study (see [Module 2's README](#) for full dataset details) so students see the exact same samples treated first as a storage/memory exercise, then as a full reproducibility exercise.

#	Script	Run as	New idea introduced
7	<code>7_Environment_Reproducibility.sh</code>	Interactive (login node)	Export/create/verify a version-pinned <code>environment.yml</code> — portable beyond this one cluster's module load system
8	<code>8_Git_Reproducibility_Workflow.sh</code>	Interactive (login node)	Initialize a Git repository, commit safely (data files excluded), and stamp a run log with the exact commit that produced it
9	<code>9_File_Format_Validation.sh</code>	Interactive (login node)	Structurally validate FASTA/FASTQ/SAM/BAM/BED/GFF/GTF/VCF files — catch truncation and malformed structure before a tool silently mis-processes them
10	<code>10_Integrated_Foundations_Capstone.sh</code>	Batch (sbatch)	Combine scripts 5, 6, 7, 8, and 9 into one run against PRJNA1518998, ending with an auto-generated <code>REPRODUCIBILITY_REFLECTION.md</code>

Why scripts 7–9 run on the login node, not via `sbatch`

Unlike scripts 1–6 and 10, scripts 7–9 are lightweight, interactive, metadata-only operations — building an environment, committing code, or running a quick text-based format check. None of them need a compute allocation, and submitting them as batch jobs would only add queue wait time for no benefit. This mirrors the same judgment call already made for `suggest_mem_from_history.sh` in the bridge extension above.

Recommended order for scripts 7–10

```
cd scripts/
bash 7_Environment_Reproducibility.sh create      # one-time: build the Conda/Mamba env
bash 8_Git_Reproducibility_Workflow.sh init      # one-time: initialize the Git repo
bash 8_Git_Reproducibility_Workflow.sh snapshot  # commit scripts + environment.yml
# ... build your srr_list.txt (see "Building your own srr_list.txt" above) ...
sbatch 10_Integrated_Foundations_Capstone.sh     # run the integrated capstone
# after it finishes:
bash 9_File_Format_Validation.sh /scratch/$(whoami)/capstone/module0/raw_reads
```

10_Integrated_Foundations_Capstone.sh also calls 9_File_Format_Validation.sh automatically on its own trimmed output as its final QC step — the manual invocation above is for validating the raw downloads separately, or for running the checks against any other directory of interest.

Nuances specific to this extension

- **module load and Conda answer different reproducibility questions.** A cluster module reproducibly gives you a tool version *on that cluster only*. `environment.yml` reproduces the same toolchain on any machine with Conda/Mamba installed — a laptop, a different center's cluster, or a cloud VM. Well-built HPC pipelines typically use both: modules for cluster-provided compilers, Conda for the bioinformatics toolchain.
- **A Git commit only proves what it claims if the working tree was clean at run time.** `8_Git_Reproducibility_Workflow.sh` stamp deliberately warns rather than silently stamping when uncommitted changes are present — always check the `git_dirty` line before trusting a logged commit hash.
- **Format validation is structural, not biological.** Scripts 9 and 10 catch "the file is broken," not "the file is correct." A FASTQ file can pass every structural check and still contain a mislabeled sample — format checks are a floor, not a ceiling, the same relationship FastQC (script 1) has to actual biological validity.
- **The reflection template is a starting point, not a substitute for thinking.** `10_Integrated_Foundations_Capstone.sh` auto-fills every fact it can measure (commit hash, environment presence, samples processed), but the genuinely reflective questions in `REPRODUCIBILITY_REFLECTION.md` (what breaks for a labmate starting from zero, what depends on unrecorded cluster-specific defaults, data licensing/privacy) still require a human answer.

Curriculum coverage in this module

Curriculum facet	Where it's covered
Linux/shell, pipes, loops, logs	Scripts 1–6
File formats (FASTA/FASTQ/SAM/BAM/BED/GFF/GTF/VCF)	Script 9
Conda/Mamba environments	Script 7
Git version control	Script 8
Compute environments (storage, memory, job scheduling)	Scripts 3–6
Responsible practice	Script 9 (validate before trusting) + the Responsible Practice Checklist below
Portfolio artifact (pipeline + env spec + README + checks + log + outputs + reflection)	Script 10

Responsible Practice Checklist

Before submitting this module's portfolio artifact, confirm:

- ☐ The dataset used (PRJNA1518998, or an alternative you selected) is public and its license/terms of use permit reuse in coursework.
- ☐ No run log, commit, or output in the repository contains personally identifiable information — not applicable to PRJNA1518998's mouse samples, but check explicitly if you substitute a human-subjects dataset.
- ☐ `.gitignore` is in place and no raw sequencing data (FASTQ/BAM/VCF/SRA) was ever committed — verify with `git log --all --stat -- '*.fastq*'` returning nothing.
- ☐ `REPRODUCIBILITY_REFLECTION.md` was completed by hand, not left as only the auto-generated template.